Deep Learning-based surrogate models for parametrized PDEs: handling geometric variability through graph neural networks

Nicola Rares Franco Stefania Fresca Filippo Tombari
Andrea Manzoni

[1] MOX, Department of Mathematics, Politecnico di Milano, Milan, Italy

Abstract

Mesh-based simulations play a key role when modeling complex physical systems that, in many disciplines across science and engineering, require the solution of parametrized time-dependent nonlinear partial differential equations (PDEs). In this context, full order models (FOMs), such as those relying on the finite element method, can reach high levels of accuracy, however often yielding intensive simulations to run. For this reason, surrogate models are developed to replace computationally expensive solvers with more efficient ones, which can strike favorable trade-offs between accuracy and efficiency. This work explores the potential usage of graph neural networks (GNNs) for the simulation of time-dependent PDEs in the presence of geometrical variability. In particular, we propose a systematic strategy to build surrogate models based on a data-driven time-stepping scheme where a GNN architecture is used to efficiently evolve the system. With respect to the majority of surrogate models, the proposed approach stands out for its ability of tackling problems with parameter dependent spatial domains, while simultaneously generalizing to different geometries and mesh resolutions. We assess the effectiveness of the proposed approach through a series of numerical experiments, involving both two- and three-dimensional problems, showing that GNNs can provide a valid alternative to traditional surrogate models in terms of computational efficiency and generalization to new scenarios. We also assess, from a numerical standpoint, the importance of using GNNs, rather than classical dense deep neural networks, for the proposed framework.

Short summary Geometric variability is a major obstacle in surrogate modeling, as classical approaches, such as the reduced basis method, can account for such degree of complexity only under severe simplifications, in an intrusive way, featuring remarkable computational costs. In this paper, we propose the use of graph neural networks to efficiently evolve dynamical systems defined on different domains and geometries. The networks are trained on a collection of trusted samples, obtained through accurate numerical simulations, and are shown to be capable of generalizing to unseen geometries without loss of accuracy. Despite assessed on a series of simplified test cases, numerical results suggest that the proposed approach can pave a new way for handling geometric variability in surrogate modeling, potentially leading to novel methodologies capable of combining GNNs and classical techniques.

1 Introduction

Thanks to accurate and reliable numerical simulations, we are now able to simulate, monitor and forecast very complex physical phenomena such as those arising in computational physics, biology and engineering. However, when it comes to many-query applications, such as, e.g., optimal control and uncertainty quantification tasks, the elevated computational cost constitutes a major limitation that hinders the effective potential of numerical simulations.

As already explored by several researchers, one way to overcome this complexity is to rely on *surrogate models*: suitable emulators that are capable of replicating the outputs of classical PDE solvers - thereby referred to as Full Order Models (FOM) - at a reduced computational cost. This practice is also known as Reduced Order Modeling (ROM). As of today, domain practitioners can count on a very large number of ROM techniques, each with its own advantages and limitations. Just to mention some of them, these include: intrusive and non-intrusive projection-based ROMs [30, 37, 21, 22, 15, 16], which can effectively tackle diffusive problems, especially in the case of affinely parametrized operators; adaptive methods based on, e.g., ROM augmentation [7, 20], clustering [19], interpolation [1] or space-time splittings [32], which are particularly suited for modeling shock waves, Hamiltonian systems, etc.; nonlinear reduction techniques based on, e.g., spectral submanifolds [6, 27] and library representations [4], which can provide users with solid theoretical guarantees; Deep-Learning based ROMs (DL-ROMs) relying on deep autoencoders, which, if provided with enough data, can address both stationary and time-dependent problems, even in the presence of severe nonlinearities and singular behaviors [12, 10, 13, 5, 8, 9, 38].

All these approaches are grounded on a common assumption, that is: the underlying FOM must be identified once and for all, with a fixed spatial discretization and a precise number of degrees of freedom (dofs) N_h. This fact, however, poses a major limitation when having to deal with PDEs defined over parametrized domains.

Assume for instance that the governing equations depend on a vector of geometrical parameters $\boldsymbol{\mu}$, which can affect the shape and the configuration of the underlying spatial domain $\Omega = \Omega_{\mu}$. Then, at the discrete level, each $\boldsymbol{\mu}$ instance will correspond to a suitable high-fidelity mesh $\mathcal{M}^h_{\mu}$ entailing N^h_{μ} dofs. The issue, here, is that as soon as we change the value of the geometrical parameters, say from $\boldsymbol{\mu}$ to $\boldsymbol{\mu}' \neq \boldsymbol{\mu}$, the total number of dofs might change, $N^h_{\mu'} \neq N^h_{\mu}$; furthermore, in general, even if $N^h_{\mu'} = N^h_{\mu}$, we will not be able to match the dofs in the two meshes. For projection-based ROMs, this makes the construction of a unique projection matrix $\mathbf{V} \in \mathbb{R}^{N_h \times n}$ impossible; similarly, we cannot rely on naive DL-ROMs as these would require the construction of an autoencoder network $\Psi \circ \Psi'$ with $\Psi' : \mathbb{R}^{N_h} \to \mathbb{R}^n$ and $\Psi' : \mathbb{R}^n \to \mathbb{R}^{N_h}$. Local techniques based on clustering algorithms can partially resolve this issue by providing a different projector $\mathbf{V}_i$ for each parametric instance $\boldsymbol{\mu}_i$ observed in the so-called *offline stage*, with $i = 1, \ldots, q$. However, this approach would incur in severe limitations during its *online* usage, as the resulting ROM would not be applicable whenever a new parametric instance $\boldsymbol{\mu} \notin \{\boldsymbol{\mu}_i\}_{i=1}^q$ is given. Similar issues are encountered when dealing with FOMs that use mesh-adaptive strategies, even if the geometry is kept fixed.

The purpose of this work is to overcome these limitations by relying on Graph Neural Networks (GNNs), thus providing a flexible approach to surrogate modeling that is capable of handling geometric variability and generalizing to unseen geometries. The idea is inspired by the recent successes of GNNs in scientific applications [44, 45, 23] and shares some similarities, that we discuss below, with other recent works.

1.1 GNNs in surrogate and reduced order modeling

GNNs are a particular class of neural network architectures that were originally proposed as a way to handle statistical data defined over graphs [42, 3]. To simplify, given a (directed) graph $G = (V, E)$ with vertices V and edges $E \subseteq V \times V$, a GNN is a computational unit that can receive a set of node features at input, $\boldsymbol{v} : V \to \mathbb{R}^l$, and return a corresponding set of node features at output $\boldsymbol{v}' : V \to \mathbb{R}^{l'}$. The same GNN unit can process data coming from different graphs. The only restrictions are: i) the size of the input and output features, l and l', respectively; ii) the fact that each input-output pair must be defined over the same graph.

This is possible because, differently from other architectures such as dense deep feed forward networks (DNNs), GNNs adopt a local perspective: information is processed at the nodal level through a combination of message-passing steps (communication of nearby nodes) and aggregation routines. This added flexibility makes GNN capable of handling data defined over different graphs and, eventually, provides them with the ability to generalize over unseen geometries. In the Deep-Learning literature, this fact is known as *relational inductive bias*. In general, the term *inductive bias* refers to the ability of a learning algorithm to prioritize one solution (or interpretation) over another, independently of the observed training data, and it can express (explicitly or implicitly) assumptions about either the data-generating process or the space of solutions[3]. In the case of GNNs, the implicit assumption is that the output of a given neuron is primarily affected by its neighbouring neurons (thus the term *relational*), so that local effects are stronger than global ones.

Our idea for the present work is to exploit the capabilities of GNNs in order to learn a nonintrusive data-driven time-stepping scheme for evolving high dimensional parameter dependent dynamical systems. To this end, we interpret discrete FOM solutions

$$\mathbf{u}_{\mu} = \left[u_{\mu,1}, \ldots, u_{\mu,N^h_{\mu}}\right]^T \in \mathbb{R}^{N^h_{\mu}}$$

as collections of nodal features $\boldsymbol{u}_{\mu} : V_{\mu} \to \mathbb{R}$, were

$$V_{\mu} = \left\{\mathbf{x}_{\mu,i}\right\}_{i=1}^{N^h_{\mu}}$$

are the vertices of the underlying mesh (sorted coherently with the FOM dofs), so that

$$\boldsymbol{u}_{\mu}\left(\mathbf{x}_{\mu,i}\right) := u^{(i)}_{\mu}.$$

Then, this graph-mesh equivalence allows us to construct a GNN module that can evolve discrete solutions defined over different meshes (and different domains).

This work finds its main inspiration in a recent contribution by Pfaff et al.[34], where the authors propose a GNN architecture for learning mesh-based simulations in a time-dependent framework. Our purpose is to transpose their ideas to the realm of ROM for parametrized PDEs, and to propose a systematic approach for handling geometric variability. To this end, we shall adopt a purely mathematical perspective, as to convey the overall idea in the language that ROM practitioners are mostly familiar with.

Nonetheless, aside from the surrounding framework and the mathematical formalism, our proposal also features a few practical differences with respect to the work by Pffaf et al., namely: i) the introduction of *global features*, which we use to extend the overall approach to nonautonomous systems and, possibly, to PDEs that depend both on physical and geometrical parameters; ii) the definition of the loss function, which we complement with an additional term concerning the approximation of the time-derivative; iii) an explicit superimposition of a Runge-Kutta-like time-stepping scheme.

In this sense, our work is much closer to the one by Pegolotti et al.[33], where the authors explore the use of GNNs for reduced order modeling of cardiovascular systems. Still, their framework remains quite different from ours as they only consider a fixed number of possible geometries, thus not allowing for a continuous parametrization, and they focus on a specific physical system. A more flexible use of GNNs is found in the recent contribution by Gladstone et al. [14], where a similar paradigms is exploited to surrogate classical PDE solvers. Their analysis, however, is limited to time-independent PDEs and does not transfer to dynamical systems.

Finally, for what concerns surrogate and reduced order modeling, we mention that some authors are also exploring the integration of GNNs together with ROM techniques: see, e.g., the GCA-ROM, a GNN-variation of the DL-ROM approach recently proposed by Pichi et al.[35] Nonetheless, these techniques are extremely different with respect to our proposal, as, in order to tackle both stationary and time-dependent PDEs, they neglect the dynamical nature of the system, that is: they treat time as an additional parameter, thus ignoring the Markovian structure that characterizes the majority of evolution equations.

1.2 Outline of the paper

The paper is organized as follows. First, in Section 2, we formally introduce the problem of surrogate modeling for parametrized dynamical systems. Then, in Section 3, we provide the reader with the fundamental building blocks required for our construction and present the corresponding GNN architectures. We then put things into action in Section 4, where we dive into the details of the proposed approach. Finally, we devote Section 5 to the numerical experiments.

2 Modeling time-dependent PDEs

We consider a PDE system depending on a set of input parameters $\boldsymbol{\mu} \in \Theta$, where the parameter space $\Theta \subset \mathbb{R}^p$ is a bounded and closed set; in our analysis, input parameters may represent both physical and geometrical properties of the system, like, e.g., material properties, boundary conditions, or the shape of the domain itself. For the time being, however, we focus on the treatment of geometrical parameters, since the extension to the case where both physical and geometrical parameters is straightforward. Throughout the paper, we adopt a fully algebraic perspective and assume that the governing equations have already been discretized in space by means of a suitable high-fidelity approximation – which, here, is allowed to depend on $\boldsymbol{\mu}$ – such as, e.g., the finite element method. Regardless of the spatial discretization adopted, the FOM can be expressed as a nonlinear, high-dimensional parametrized dynamical system. Hence, given $\boldsymbol{\mu} \in \Theta$, we aim at solving the initial value problem:

$$\begin{cases} \dot{\mathbf{u}}_{\mu}(t) = \mathbf{f}\left(t, \mathbf{u}_{\mu}(t), \boldsymbol{\mu}\right), & t \in (0, T), \\ \mathbf{u}_{\mu}(0) = \mathbf{g}_{\mu}, \end{cases} \tag{1}$$

where $\mathbf{u}_{\mu} : [0, T) \to \mathbb{R}^{N^h_{\mu}}$ is the parametric solution to (1), while

$$\mathbf{g}_{\mu} \in \mathbb{R}^{N^h_{\mu}} \quad \text{and} \quad \mathbf{f}(\cdot, \cdot, \boldsymbol{\mu}) : (0, T) \times \mathbb{R}^{N^h_{\mu}} \to \mathbb{R}^{N^h_{\mu}}$$

are the initial condition and a - possibly nonlinear - function encoding the dynamics of the system, respectively. The FOM dimension, N^h_{μ}, is related to the finite dimensional subspaces introduced for the sake of space discretization – here $h > 0$ denotes a discretization parameter, such as the maximum diameter of the elements in the computational mesh $\mathcal{M}^h_{\mu}$; consequently, N^h_{μ} can be extremely large if the PDE problem describes complex physical behaviors and/or high degrees of accuracy are required for its solution. Furthermore, the number of degrees of freedom (dofs) of the problem may depend on the geometrical parameters contained in $\boldsymbol{\mu}$ since, by modifying their values, the number of vertices in the computational mesh $\mathcal{M}^h_{\mu}$ can vary. We thus aim at approximating the set

$$\mathcal{S} = \{\mathbf{u}_{\mu}(t) \mid \ t \in [0, T), \ \ \boldsymbol{\mu} \in \Theta \subset \mathbb{R}^p\} \subset \bigcup_{\mu \in \Theta} \mathbb{R}^{N^h_{\mu}} \tag{2}$$

of the solutions to (1) when $(t; \boldsymbol{\mu})$ varies in $[0, T) \times \Theta$, also referred to as solution manifold.

In order to numerically approximate problem (1), even at the FOM level, one must rely on suitable time-integration schemes, such as the backward differentation formulae [36]. Thus, having fixed a uniform partition of $(0, T)$ in N_t equally spaced subintervals, and by denoting with $\mathbf{u}^n$, the solution $\mathbf{u}$ at time $t^n = n\Delta t$, where $\Delta t := T / N_t$, our ultimate aim is to solve:

$$\begin{cases} \dfrac{\mathbf{u}_\mu^{n+1} - \mathbf{u}_\mu^n}{\Delta t} = \mathbf{f}\left(t^{n+1}, \mathbf{u}^{n+1}{}_\mu, \boldsymbol{\mu} \right), & n \geq 0, \\ \mathbf{u}_\mu^0 = \mathbf{g}_\mu. \end{cases} \tag{3}$$

Equation (3) requires the solution, at each time instance, of a nonlinear system depending on the input parameter vector $\boldsymbol{\mu}$ which may entail high computational times, especially when dealing with a multi-query or real-time context. To achieve computational efficiency, multi-query analysis and real-time problems must rely on suitable surrogate models which can be built according to different strategies. Motivated by this, our goal is the efficient approximation of the solution manifold in (2) by decreasing the complexity related to the solution of the FOM and preserving high level of accuracy.

In this work, we introduce a Deep Learning-based surrogate model that exploits graph neural networks (GNNs) [17] to efficiently evolve the time-discrete dynamical system in (3). Here, the use of GNNs is motivated by their unique ability of handling data defined on different graphs/meshes, which can result in extremely flexible models capable of generalizing to new, unseen, geometries and spatial resolutions (in the Deep Learning literature, this fact is usually referred to as relational inductive bias[3]). In mathematical terms, we aim at constructing a GNN architecture Φ such that:

$$\mathbf{u}_\mu^{n+1} \approx \boldsymbol{\Phi}(\mathbf{u}_\mu^n, t^n, \boldsymbol{\mu}).$$

From an abstract point of view, the above can be seen as an extension of the MeshGraphNet model as originally proposed by Pfaff et al.[34]: here, in fact, the time variable is included explicitly, which allows us to address the more general case of nonautonomous systems.

3 Graph Neural Networks

GNNs were initially conceived as an extension of convolutional neural networks (CNNs) to operate on graph-structured data and overcome their limitations in this domain. In graph theory, graphs are used to describe systems made of nodes and their connections (edges). An image can be regarded as a graph with regular and well-organized connections in the Euclidean space, where each pixel corresponds to a node in the graph. In this particular case, the aforementioned CNN architectures can exploit the peculiar structure of the graph to extract meaningful spatial features. However, these models become inapplicable as soon as the structure of the underlying graph becomes slightly more sophisticated: in practice, this fact has remarkable consequences as in many real-time applications (e.g., traffic networks, socialnetworks) data are naturally defined over general, possibly non-Euclidean, graphs.

To promote the use of Deep Learning in those applications, GNNs were developed to extract spatial features over general graphs, by inspecting neighboring nodes, with arbitrary connection in a non-Euclidean space [31, 47]. Thus, GNNs can be considered as a generalized version of CNNs over general graphs. Clearly, this generalization comes with some differences between the two architectures. For instance, while the outputs of CNNs are affected by the ordering of the pixels, the same is not true for GNNs, as the action of the latter is uniquely determined by the connectivity of the graph (in this concern, note, for instance, that the connectivity of an image remains the same even if we flip it either vertically or horizontally). Moreover, GNNs adopt a *graph-in, graph-out* architecture meaning that these model types accept a graph as input, with information loaded into its nodes and edges, and progressively transform these embeddings, without changing the connectivity of the input graph: in contrast, CNN layers usually modify the resolution of the input upon their action.

The fundamental ingredient of a basic GNN layer is the so-called *message passing* operation, which enables the aggregation of node information while leveraging the depth of the graph. More precisely, a message passing step consists of two components:

- message computation: each node creates a message to be sent to other nodes later;
- aggregation: each node aggregates the messages from the neighborhood.

This message-passing propagation can be seen as an information retrieval task from different levels of depth of the graph. A simple visualization of the message propagation is shown in Figure 1. For each node, the information comes from the neighbors. In this way, adding message-passing steps can be seen as connecting nodes that can be also far from each other.

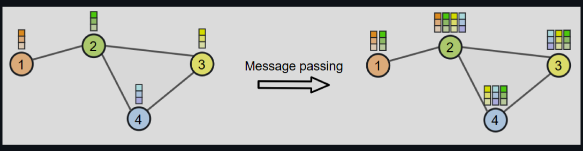

Figure 1: Message propagation and aggregation. The information is broadcasted from different levels of depth of the graph. For each node, at each message passing step,the information is collected from the neighbors and aggregated. In this way, adding message-passing steps can be seen as connecting nodes which can also be far from each other. Figure courtesy of Phillip Lippe (University of Amsterdam, QUVA lab).

Graph-based algorithms, such as, e.g, graph convolutional networks [26], GraphSage [18], graph attention networks [46], graph transformer operator [43] and interaction networks [2], differ in the way the message is computed and the aggregation is performed. In particular, depending on the chosen framework, the message-passing step may also involve the edges of the graph, where a corresponding set of *edge features* can be loaded: in the next few pages, we shall describe this situation in full mathematical detail, as it will be of key importance for our construction.

In order to perform a message-passing operation, GNNs leverage on suitable data structures for representing the topology and connectivity of the graph. In this concern, a classical choice is to exploit the edge connectivity matrix, that is, a $n_{edges} \times 2$ matrix where each row k contains the indices of the source and destination nodes of the kth edge; this allows GNNs to stash the overall topology of the graph with a memory complexity of $\mathcal{O}(n_{edges})$. Roughly speaking, this is equivalent to storing a sparse version of the adjacency matrix of the graph, which, in principle, consists of $\mathcal{O}(n_{nodes}^2)$ entries.

Before coming to our own use of GNNs for surrogate modeling, within this Section we take the chance to present some of the fundamental ingredients required for our construction. In particular, we shall describe in mathematical terms the concept of message-passing, and we shall introduce a particular GNN architecture known as the Encoder-Processor-Decoder model.

3.1 The message-passing block: formal definition

Given $l \in \mathbb{N}$, a graph-forward-pass with l hidden features is a computational unit $F = F(\boldsymbol{v}, \boldsymbol{e}, G)$ that takes as input

i) a directed graph structure, $G = (V, E)$;

ii) a collection of vertex features, $\boldsymbol{v} : V \to \mathbb{R}^l$;

iii) a collection of edge features, $\boldsymbol{e} : E \to \mathbb{R}^l$

and outputs a new collection of vertex features with l-features per node, namely

$$F(\boldsymbol{v}, \boldsymbol{e}, G) : V \to \mathbb{R}^l.$$

We think of F as an object that transforms the vertex features associated to the nodes in the graph.

In GNN architectures, a message-passing block is a particular type of graph-forward-pass routine that exploits the local structure of the input graph G, only allowing communication of nearby nodes. Specifically, a message-passing block F is comprised of two Multi-Layer Perceptron[29] (MLP) units,

$$\psi_v : \mathbb{R}^{2l} \to \mathbb{R}^l \quad \text{and} \quad \psi_e : \mathbb{R}^{3l} \to \mathbb{R}^l,$$

that completely characterize the action of F. However, in order to properly explain how the forward pass is carried out, we first need to introduce some notation. Given a graph $G = (V, E)$ and a collection of vertex features $\boldsymbol{v} : V \to \mathbb{R}^l$, we write $\boldsymbol{v}_{\text{in}}$ and $\boldsymbol{v}_{\text{out}}$ for the maps

$$\boldsymbol{v}_{\text{in}} : E \to \mathbb{R}^l \qquad \boldsymbol{v}_{\text{out}} : E \to \mathbb{R}^l$$

given by

$$\boldsymbol{v}_{\text{in}}(v_1, v_2) := \boldsymbol{v}(v_1), \quad \boldsymbol{v}_{\text{out}}(v_1, v_2) := \boldsymbol{v}(v_2),$$

respectively, where $(v_1, v_2) \in E$ represents an oriented edge going from v_1 to v_2. In other words, passing from $\boldsymbol{v}$ to $\boldsymbol{v}_{\text{in}}$ is equivalent to transferring the information from the nodes to the edges, with the convention that a given edge inherits the features from its own *source node*. Similarly, going from $\boldsymbol{v}$ to $\boldsymbol{v}_{\text{out}}$ is a way for storing the information about the *destination nodes*.

In the same spirit, it is also useful to define the dual operation, which transfers information from the edges to the nodes. More precisely, given $\boldsymbol{e} : E \to \mathbb{R}^l$, we shall write $\bar{\boldsymbol{e}}$ for the map $\bar{\boldsymbol{e}} : V \to \mathbb{R}^l$ defined as

$$\bar{\boldsymbol{e}}(v) := \sum_{(v_1, v) \in E} \boldsymbol{e}(v_1, v),$$

that is, to go from $\boldsymbol{e}$ to $\bar{\boldsymbol{e}}$, we collapse all the features corresponding to edges with the same destination node.

Lastly, we shall denote by $\oplus$ the concatenation operator. Specifically, given any two functions with a common domain, e.g., $\boldsymbol{f} : X \to \mathbb{R}^a$ and $\boldsymbol{g} : X \to \mathbb{R}^b$, we write $\boldsymbol{f} \oplus \boldsymbol{g}$ to intend the map from Ω to $\mathbb{R}^{a+b}$ given by

$$\boldsymbol{f} \oplus \boldsymbol{g}(x) := [f_1(x), \ldots, f_a(x), g_1(x), \ldots, g_b(x)],$$

where $x \in X$ is a generic input, while f_i and g_j are the ith and jth components at the output of $\boldsymbol{f}$ and $\boldsymbol{g}$, respectively.

We now have all the ingredients to rigorously define the forward-pass of a message-passing block. The action of a message-passing block F with l hidden features and computational units ψ_v, ψ_e, is defined as

$$F(\boldsymbol{v}, \boldsymbol{e}, G) = \psi_v \circ \left(\boldsymbol{v} \oplus \psi_e \circ (\boldsymbol{e} \oplus \bar{\boldsymbol{v}}_{\text{in}} \oplus \boldsymbol{v}_{\text{out}}) \right), \tag{4}$$

where, as usual, $\circ$ denotes functional composition.

In plain words, Eq. (4) states that the vertex features at output, $F(\boldsymbol{v}, \boldsymbol{e}, G)$, are obtained as follows: first, the information available in the graph vertices is transferred to the edges and concatenated with the existing features, $\boldsymbol{e} \oplus \boldsymbol{v}_{\text{in}} \oplus \boldsymbol{v}_{\text{out}}$; then, an MLP, ψ_e, is applied to the extended features to extract meaningful information; the latter, is then transferred back to the node vertices, yielding $\psi_e \circ (\boldsymbol{e} \oplus \bar{\boldsymbol{v}}_{\text{in}} \oplus \boldsymbol{v}_{\text{out}})$. These hidden features – which now live of the graph vertices – are then appended to the original ones and later fed to a terminal MLP block, here given by ψ_v.

In general, we remark that the action of a message-passing step $(\boldsymbol{v}, \boldsymbol{e}, G) \mapsto F(\boldsymbol{v}, \boldsymbol{e}, G)$ is nonlinear because of the two MLPs, ψ_v and ψ_e, entering the pipeline.

Remark 1 *Note that the operations* $\mathbf{v} \mapsto \mathbf{v}_{in}$, $\mathbf{v} \mapsto \mathbf{v}_{out}$ *and* $\mathbf{e} \mapsto \bar{\mathbf{e}}$, *require an exact knowledge of the graph structure G. Here, this fact is left implicit to keep the notation lighter.*

Remark 2 *In the literature, GNNs have been defined in several ways. One major difference lies in that some authors only talk about «node features», without contemplating the existence of «edge features». Here, we are adopting one of the most recent formulation of GNNs, as proposed by Battaglia et al.[3] Nonetheless, we believe that finding connections between different definitions can enhance practical understanding. To this end, we mention that in the classical formulation by Scarselli et al. (that is, without edge features), a major role is played by the «aggregation step», in which information coming from neighbouring nodes is collapsed onto a single value, e.g. via summation (see Equation 3 in the original work by Scarselli et al. [42]). Here, the same effect can be obtained via* $\mathbf{v} \mapsto \mathbf{v}_{in} \bar{\oplus} \mathbf{v}_{out} - (\mathbf{n} - 1) \cdot \mathbf{v}$, *where* $\mathbf{n} : V \to \mathbb{N}$ *is a feature map that returns the connectivity of each node, whereas* $\cdot$ *stands for pairwise multiplication.*

3.2 The Encoder-Processor-Decoder model

The Encoder-Processor-Decoder model is a powerful GNN-based architecture that can process mesh-based data [3, 34, 41]. More precisely, the latter accepts as input:

i) a directed graph $G = (V, E)$ associated to some mesh $\mathcal{M}$ embedded in a suitable ambient space $\mathbb{R}^d$, so that $V \subset \mathbb{R}^d$;

ii) an input signal defined over the mesh vertices, namely $\boldsymbol{u} : V \to \mathbb{R}^q$;

iii) a global feature vector, $\xi \in \mathbb{R}^s$, describing a given nonspatial property of the system (e.g., time).

Then, the output of such a model is a new signal $\boldsymbol{u}' : V \to \mathbb{R}^{q'}$ defined over the given mesh.

As the name suggests, the Encoder-Processor-Decoder model is comprised of three modules, which we explain in detail below. These are all characterized by a common hidden-dimension, $l \in \mathbb{N}$, which we assume to be fixed hereon.

3.2.1 Encoder module

The encoder module is used to preprocess the input data and return a collection of hidden features defined, respectively, over the graph vertices $\mathcal{E}_v = \mathcal{E}_v(\boldsymbol{u}, \xi, G)$ and the edge vertices $\mathcal{E}_e = \mathcal{E}_e(G)$. The two are obtained as follows.

The node features $\mathcal{E}_v(\boldsymbol{u}, \xi, G)$, are computed by combining a fixed nonlearnable transformation together with an MLP unit $\Psi_{\mathcal{E}}^v : \mathbb{R}^{q+s+1} \to \mathbb{R}^l$ that maps onto the hidden-state space. The former has the purpose of expanding the node features with information coming from the global variables, ξ, and the graph G. More precisely, let $\boldsymbol{b}_G : V \to \{0,1\}$ be a flag for those nodes that lie on the boundary of the mesh, i.e., $\boldsymbol{b}_G(v) = 1$ if and only if v is a boundary vertex. Then, the action of $\mathcal{E}_v$ reads

$$\mathcal{E}_v(\boldsymbol{u}, \xi, G) := \Psi^v_{\mathcal{E}} \circ (\boldsymbol{u} \oplus \xi \oplus \boldsymbol{b}_G), \tag{5}$$

so that $\mathcal{E}_v(\boldsymbol{u}, \xi, G) : V \to \mathbb{R}^l$. Here, with little abuse of notation, we have identified the vector ξ with a constant map defined over V. As we mentioned, the preliminary transformation $\boldsymbol{u} \mapsto \boldsymbol{u} \oplus \xi \oplus \boldsymbol{b}_G$ is nonlearnable and has the sole purpose of augmenting the nodal features; conversely, the MLP unit introduces a learnable block that is optimized during training.

The edge features $\mathcal{E}_e(G)$ are computed following similar ideas. First, a set of nonlearnable features $e_G : E \to \mathbb{R}^{d+1}$ is extracted starting from the mesh coordinates. This is achieved by letting

$$e_G(\mathbf{x}_1, \mathbf{x}_2) := \left[\frac{x_1^{(1)} + x_2^1}{2}, \ldots, \frac{x_1^{(d)} + x_2^d}{2}, |\mathbf{x}_1 - \mathbf{x}_2| \right]$$

where $(\mathbf{x}_1, \mathbf{x}_2) \in E$. In other words, e_G maps each edge to a vector containing the coordinates of its midpoint together with the edge length. These preliminary features are then fed to an MLP $\Psi^e_{\mathcal{E}} : \mathbb{R}^{d+1} \to \mathbb{R}^l$, i.e.

$$\mathcal{E}_e(\boldsymbol{u}, \xi, G) := \Psi^e_{\mathcal{E}} \circ e_G, \tag{6}$$

which returns the encoded edge features.

3.2.2 Processor module

The encoded features, $\boldsymbol{v} := \mathcal{E}_v(\boldsymbol{u}, \xi, G)$ and $\boldsymbol{e} := \mathcal{E}_e(G)$, are then elaborated by a GNN-based unit, called the processor $\mathcal{P}$. The latter consists of m message-passing-blocks, $F_1, \ldots, F_m$, each one acting as in (4). More precisely, the output of the processor module is given by

$$\mathcal{P}(\boldsymbol{v}, \boldsymbol{e}, G) := F_m(\boldsymbol{h}_m, \boldsymbol{e}, G), \tag{7}$$

where

$$\begin{cases} \boldsymbol{h}_1 = \boldsymbol{v} \\ \boldsymbol{h}_{j+1} = F_j(\boldsymbol{h}_j, \boldsymbol{e}, G) \; j = 1, \ldots, m-1, \end{cases}$$

so that the final output is obtained by applying the blocks $F_1, \ldots, F_m$ iteratively. We highlight how each message-passing-step transforms the node features but not the edge features. We also point out that, since a single message-passing-block allows neighbouring nodes to exchange information, a processor module with m units allows communication between nodes that are m edges faraway. Thus, by changing the number of message-passing-steps, one can move from local to nonlocal transforms (with the latter possibly being more expressive). However, we must also mention that large values of m may give rise to oversmoothing phenomena[40], reason for which, in practice, a suitable compromise is required.

3.2.3 Decoder module

In the end, the processor outputs some collection of node features $\boldsymbol{v}' := \mathcal{P}(\boldsymbol{v}, \boldsymbol{e}, G)$, with $\boldsymbol{v}' : V \to \mathbb{R}^l$. At this point, a terminal module, called the decoder, is exploited to recover the desired output. Here, we assume the latter to be consistent with the input signal $\boldsymbol{u}$, and thus consist of q nodal features. In practice, this is achieved by relying on a suitable MLP unit $\Psi_{\mathcal{D}} : \mathbb{R}^l \to \mathbb{R}^q$, that transforms the original l features onto the q desired ones. In other words, the decoder module operates nodewise, and its action can be written as

$$\mathcal{D}(\boldsymbol{v}') := \Psi_{\mathcal{D}} \circ \boldsymbol{v}'. \tag{8}$$

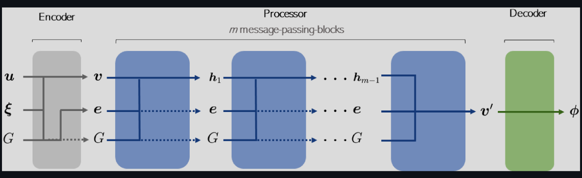

Figure 2: Visual representation of the Encoder-Processor-Decoder model, Section 3.2. Rigid arrows represent algorithmic computations (learnable and nonlearnable), while dashed arrows act as pointers (no computation implied). In gray, the encoder module, Eqs. (5)-(6); in blue, the message-passing-blocks defining the processor unit, Eq. (7); in green, the decoder module, Eq. (8).

3.2.4 Overall architecture

To summarize, the computational workflow of an Encoder-Processor-Decoder model reads

$$\Phi(\boldsymbol{u}, \xi, G) := \mathcal{D}(\mathcal{P}(\mathcal{E}_v(\boldsymbol{u}, \xi, G), \mathcal{E}_e(G), G)). \tag{9}$$

The reader can also find a visual depiction of Eq. (9) in Figure 2. Since the notation might be troublesome, we remark that the output of the Encoder-Processor-Decoder, $\boldsymbol{\phi} := \Phi(\boldsymbol{u}, \xi, G)$, is nothing but a collection of q-dimensional node features $\boldsymbol{\phi} : V \to \mathbb{R}^q$.

Remark 3 *In the literature of surrogate and reduced order modeling, the words «Encoder» and «Decoder» are often associated to the concept of dimensionality reduction, where the two objects operate, respectively, to achieve data compression and reconstruction. Here, however, the meaning is completely different. The encoder module acts as a feature extractor and, in general, may increase the dimension of the input; conversely, the decoder module is used to map the nodal feature space onto the nodal output space (usually decreasing the dimension at each node), so to recover the quantities of interest. While the notation might be confusing to some of the readers, we have decided to stick to the one adopted by the GNN community* [34, 41].

4 Application to surrogate modeling of parametrized PDEs

Our goal is to predict an approximate solution $\tilde{\mathbf{u}}$ at time t^{n+1}, given the state of the system at time t^n, for each node $i = 1, \ldots, N^h_\mu$ of the computational mesh $\mathcal{M}^h_\mu$, that is

$$\Phi(\mathbf{u}^n_\mu, t^n, \boldsymbol{\mu}) \approx \mathbf{u}^{n+1}_\mu.$$

Inspired by the general form of explicit Runge-Kutta methods, we model the time stepping scheme Φ by letting

$$\Phi(\mathbf{v}, t^n, \boldsymbol{\mu}) := \mathbf{v} + \Delta t \tilde{\Phi}(\mathbf{v}, t^n, \mathcal{M}^h_\mu), \tag{10}$$

where $\tilde{\Phi}$ is a GNN architecture based on the Encoder-Processor-Decoder paradigm. Here, with little abuse of notation, we are identifying the computational mesh $\mathcal{M}^h_\mu$ with its underlying graph $G_\mu = (V_\mu, E_\mu)$, and the dof vector $\mathbf{u}^n_\mu \in \mathbb{R}^{N^h_\mu}$ with its corresponding vertex feature map $\boldsymbol{u}^n_\mu : V_\mu \to \mathbb{R}$. We also remark that in Eq. (10), the GNN model is made aware of the current time instant: in fact, according to our notation in Section 3.2, the latter is being interpreted as a global feature $\xi = t^n$. Therefore, the proposed approach aims at modeling the time-stepping scheme using an Encoder-Processor-Decoder architecture, $\tilde{\Phi}$, that incorporates both the nodal features at a specific time instance t^n, as well as the geometrical features of the mesh. The architecture aggregates information from neighboring nodes, processes it, and decodes the system solution at time t^{n+1}. This allows us to evaluate Equation (10) independently of the number of dofs, while simultaneously accounting for the graph structure of the mesh. In particular, this makes it possible to train the model using a variety of different geometries and subsequently predict solutions for new meshes that were not included in the training data.

This flexibility is guaranteed by the relational inductive bias of GNNs, which ultimately comes the message-passing paradigm: the model first computes the messages between neighboring nodes, and then performs a suitable aggregation of the information. In contrast, models based on FFNNs and CNNs are constrained by the number of nodes in the computational mesh, which prevents them from generalizing to different domains. The same issue is also encountered by other architectures, such as Mesh-Informed Neural Networks[11] (MINNs): in fact, even though MINNs can handle very complicated geometries at reduced training times, their implementation requires fixing the shape of the spatial domain and the resolution of the space discretization. In this sense, the additional flexibility provided by GNNs is extremely valuable.

4.1 Training and testing algorithms

From an operational point of view, the GNN model $\tilde{\Phi}$ in (10) is trained on a suitable dataset of FOM solutions that serves as a ground truth reference. More precisely, after having constructed and initialized the GNN model, we exploit the FOM solver to generate a collection of training snapshots,

$$\{\boldsymbol{\mu}_i, \mathbf{u}^0_{\mu_i}, \ldots, \mathbf{u}^{N_t}_{\mu_i}\}_{i=1}^{N_{\text{train}}},$$

containing a total of N_{train} different trajectories, each corresponding to a different geometrical configuration $\boldsymbol{\mu}$. For the sake of simplicity, we assume that all the trajectories consist of N_t snapshots in time: however, this assumption is not fundamental to our construction and it can easily dropped.

Let θ be the vector collecting all the parameters of the GNN module. To emphasize the dependency of the latter on θ, let us write $\tilde{\Phi}_\theta$ in place of $\tilde{\Phi}$. We train the GNN architecture by minimizing the loss function below

$$\begin{aligned} \mathcal{L}(\theta) = \; & c w_1 \sum_{i=1}^{N_{\text{train}}} \sum_{n=0}^{N_t - 1} |\mathbf{u}^{n+1}_\mu - \mathbf{u}^n_\mu - \Delta t \tilde{\Phi}_\theta(\mathbf{u}^n_{\mu_i}, t^n, \mathcal{M}^h_{\mu_i})|^2 + \\ & c w_2 \sum_{i=1}^{N_{\text{train}}} \sum_{n=0}^{N_t - 1} |\dot{\mathbf{u}}^n_\mu - \tilde{\Phi}_\theta(\mathbf{u}^n_{\mu_i}, t^n, \mathcal{M}^h_{\mu_i})|^2, \end{aligned} \tag{11}$$

where $c = 1 / N_{\text{train}} N_t$ is a normalizing factor, whereas w_1 and w_2 are suitable hyperparameters to be tuned manually. The term $\dot{\mathbf{u}}^n_\mu$, instead, refers to a suitable finite-difference approximation of the ground truth time-derivative (e.g., computed by relying either on the forward or backward formulae). The loss function in (11) is made of two contributes: the first one, quantifies the error of the time-stepping scheme after a single iteration; the second one, instead, links the FOM derivative with the output of the GNN model. In particular, we do not rely on full rollouts or any other form of recursive training: this allows us to fully exploit the capabilities of GPUs tensor calculus and mitigate memory usage.

Clearly, the downside to this is that, even after a successful training, our GNN model might be subject to error propagation when advancing in time multiple times. To limit this issue and ensure robust rollouts, we exploit the following strategies. At each epoch, that is, at each iteration of the optimization routine:

- we do not directly optimize (11), but rather rely on randomly selected mini-batches;
- we gitter the input data with random Gaussian noise, as to limit the sensitivity of $\tilde{\Phi}$ and to enhance the stability of the rollouts at prediction;

In practice, the optimization of the GNN model is carried out by relying on back-propagation [39] and ADAM [25], with a variable learning rate that we decrease by a factor $\gamma > 0$ after a specific number of epochs (see Algorithm 1). In general, the training of the GNN model can be carried out iteratively until a stopping criterion is met. For instance, one may simply stop the training after a predefined number of epochs, see, e.g., Algorithm 1.

Input: network $\tilde{\Phi}$; timestep Δt; a list of N_{train} training trajectories $\mathbf{U}$ (each of length N_t); a list of edge connectivity matrices $\mathbf{E}$; a list of edge features matrices $\mathbf{W}$; a list of inner nodes $\mathbf{I}$; learning rate ν; decay factor γ; number of training epochs `epochs`; batch size N_b; noise variance σ^2.

Output: optimal model parameters θ^*.

Algorithm 1 Training Algorithm

```
epoch = 0.
Randomly initialize $\theta^0$.
while epoch < $max\_epoch$ do
    Create the list $indices = [1, \ldots, N_{train}]$ and shuffle it randomly.
    for $sim$ in $indices$ do
        $\mathbf{U}_{sim} = \mathbf{U}[sim]$, $\mathbf{U}_{sim} \in \mathbb{R}^{N_t \times N_h \times q}$ where $N_t$ is the total number of time instanc
        $\mathbf{E}_{sim} = \mathbf{E}[sim]$, $\mathbf{E}_{sim} \in \mathbb{R}^{N_{edges} \times 2}$.
        $\mathbf{W}_{sim} = \mathbf{W}[sim]$, $\mathbf{W}_{sim} \in \mathbb{R}^{N_{edges} \times N_e}$ where $N_e$ is the number of edge features.
        $\mathbf{I}_{sim} = \mathbf{I}[sim]$, $\mathbf{I}_{sim} \in \mathbb{R}^{N_h}$ with $\mathbf{I}_{sim}[i] = 1$ if node $i$ is an inner node, 0 otherwis
        $b = 0$.
        while $b < N_t$ do
            $\mathbf{U}_b = U_{sim}[b : b + N_b]$.
            Create noise tensor $\boldsymbol{\Sigma} = \sigma \mathbf{Z}$ where $\mathbf{Z} \in \mathbb{R}^{N_b \times N_i \times q}$ is a random tensor with $N$
            Initialize $\mathbf{U}_{noise} = \mathbf{U}_b$.
            $\mathbf{U}_{noise}[:, I_{sim}] \; += \; \Sigma$.
            Calculate target derivative $\mathbf{U}_{dot} = (\mathbf{U}_b[1:] - \mathbf{U}_{noise}[:-1]) / \Delta t$.
            Make a forward pass through the network $\tilde{\Phi}(\mathbf{U}_{noise}, \mathbf{E}_{sim}, \mathbf{W}_{sim})$.
            Calculate network solution $U_{net} = \mathbf{U}_{noise}[:-1] + \Delta t \tilde{\Phi}$.
            Calculate training loss $\mathcal{L}_b$.
            Back-propagation through the net and parameters update: $\theta^1 = \text{ADAM}(\nu, \theta^0$
            $\theta^0 = \theta^1$.
            $b \leftarrow b + N_b$.
        end while
    end for
    if mod(epoch, 500) = 0 then
        Reduce learning rate by a factor $\gamma$.
    end if
    epoch ← epoch + 1.
end whilePick the last weights updated $\theta^1$.
```

Once the model has been trained and a suitable vector of parameters θ^* has been selected, the GNN is fully operational. That is, given any configuration of the geometric parameters $\boldsymbol{\mu}$ and any initial condition $\mathbf{u}_{\mu}^{0}$, we can exploit the GNN model and (10) *online* to evolve the system iteratively and produce a complete rollout $\{\tilde{\mathbf{u}}_{\mu}^{n}\}_{n=0}^{N_t}$, where

$$\begin{cases} \tilde{\mathbf{u}}_{\mu}^{n+1} = \Phi(\tilde{\mathbf{u}}_{\mu}^{n}, t^n, \boldsymbol{\mu}) \; n \geq 0 \\ \tilde{\mathbf{u}}_{\mu}^{0} := \mathbf{u}_{\mu}^{0}. \end{cases} \tag{12}$$

Here, to further improve stability, one may also enforce any external constraint, such as Dirichlet boundary conditions, at each time iteration.

To test the quality of the GNN surrogate, we compare its predictions with those of the FOM for a set of new parameter instances. In particular, differently from the training stage, we now compare the overall trajectories and use the GNN to produce full rollouts of the solution.

Quantitatively, we compute the prediction error as the relative MSE (RMSE) error between the network prediction and the ground truth solution:

$$RMSE(\tilde{\mathbf{u}}_{\mu}^{1}, \dots, \tilde{\mathbf{u}}_{\mu}^{N_t}, \mathbf{u}_{\mu}^{1}, \dots, \mathbf{u}_{\mu}^{N_t}) = \frac{1}{N_t} \sum_{n=1}^{N_t} \frac{|\tilde{\mathbf{u}}_{\mu}^{n} - \mathbf{u}_{\mu}^{n}|^2}{|\mathbf{u}_{\mu}^{n}|^2}, \tag{13}$$

where the GNN rollout is obtained as in (12).

5 Numerical experiments

In this Section, we assess the capabilities of the proposed approach over three advection-diffusion problems of increasing complexity:

- a scalar diffusion in a 2D square with a circular obstacle and a time-varying advection term;
- a 2D Stokes flow in proximity of a bump;
- a 3D Stokes flow around a cylinder.

All the examples are characterized by parameter dependent spatial domains, where a given obstacle is allowed to move across the domain, with possible changes in terms of shape and dimension. In this way, we can effectively test the ability of GNNs in handling geometric variability.

5.1 Advection-Diffusion problem in a square domain with a circular obstacle

To start, we consider the following advection-diffusion problem:

$$\begin{cases} \dfrac{\partial u}{\partial t} - D\Delta u + \mathbf{b} \cdot \nabla u = 0 & \text{in } \; \Omega \times (0, T] \\ u(x, y) = (x-1)^2 + (y-1)^2 & \text{on } \; \partial\Omega \times (0, T] \\ u_0(x, y) = (x-1)^2 + (y-1)^2 & \text{in } \; \Omega, \end{cases} \tag{14}$$

where $\Omega = (0,1)^2 \setminus C$, with

$$C = \{(x, y): \; (x - c_x)^2 + (y - c_y)^2 \leq (0.15)^2\}.$$

Here, we set $T = 2$, $D = 0.1$ and $\mathbf{b}(t) = [1 - t, 1 - t]$. In particular, due to the time-varying convection field, $\mathbf{b}$, the resulting dynamical system can be regarded as nonautonomous. In our simulations, we parametrize the center of the circle as

$$\boldsymbol{\mu} = (c_x, c_y) \in \Theta := \{(x, y) : 0 < x < 1, \;\; y \geq 0.5\},$$

which we let vary as we generate the training data. In agreement with Equations (1)-(3), the ground truth FOM simulations of Problem (14) are obtained by first discretizing in space via P1 Continuous Galerkin Finite Elements, and then in time using the Backward Euler Method. The time step chosen is $\Delta t = 0.02$, resulting in 101 time snapshots for each simulation. We also mention that, following our notation in Section 3, we have $q = 1$, as the solutions to (14) are scalar fields.

5.1.1 Problem data

We collected a dataset composed of 100 simulations, each obtained for a different position of the center of the obstacle, with a number of mesh nodes varying from 770 to 790. The training set is composed of 80 randomly selected simulations, while the remaining 20 are kept for testing.

For what concerns the design of the GNN architecture and its training, we have reported a synthetic overview in Table 1. In particular, in this case, we adopt a simplified loss function that only features the approximation of the time-derivative; in other words, we set $w_1 = 0$ and $w_2 = 1$ in Equation (11).

Problem	**MP steps**	**MLP layers**	l	**Activ**	**Epochs (max)**	**Batch size**	**lr**	γ	σ^2	w_1	w_2
Adve-diff	12	2	32	SiLU	1500	25	10^{-3}	0.1	10^{-6}	0	1
Stokes 2D	18	2	32	SiLU	3000	25	10^{-3}	0.1	10^{-6}	0.5	0.5
Stokes 3D	15	2	32	SiLU	2000	25	10^{-4}	0.1	10^{-5}	0.5	0.5

Table 1: GNN architecture and training hyperparameters for the three case studies. MP = message passing, MLP layers = (common) depth of all the MLP units in the Encoder-Processor-Decoder pipeline, l = (local) feature space dimension, lr = learning rate, γ = learning rate decay factor (applied every 500 epochs), σ^2 = noise variance, w_i loss function weights. SiLU = Sigmoid weighted Linear Unit, $x \to x \,/\, (1 + \exp(-x))$.

5.1.2 Numerical results

Results are in Table 2. As we can see, all the predictions RMSEs are of order 10^{-3} to 10^{-4}. Moreover, our model outperforms significantly the ground truth solver in the simulation time at testing stage. The dynamic of the problem is well predicted and no propagation errors are spotted. Hence, our model appears capable of solving problems concerning evolutionary PDEs, in that it can approximate multiple time steps in a stable way.

Still, it is worth looking at some of the simulations obtained during the testing phase, as to further appreciate the ability of the proposed approach in handling different geometric configurations. For instance, Figures 3 and 4 show two different GNN rollouts corresponding to two different positions of the obstacle. Despite these trajectories being different from the ones seen during training, the model manages to capture all the main features characterizing the solutions, such as the behavior near the obstacle and the direction of propagation.

We highlight that a GNN-based approach follows a *local-to-global* paradigm, first processing information at the node level (encoder), and then aggregating the output at the neighbour level (processor). Clearly, the lack of smoothness in PDE solutions can pose some challenges, as GNNs are known to struggle with capturing such properties. In this sense, it is not surprising to see that the prediction in Figure 4 is worse than the one in Figure 3. In fact, in the former case, the obstacle is closer to the corner of the spatial domain. Of note, we mention that the trajectory in Figure 4 is actually the worst across the whole test set.

It is also interesting to see that the prediction error exhibits an oscillating trend. In fact, after a first increase, the accuracy appears to improve ($t = 0.5$ vs $t = 1.00$), which is most likely caused by the presence of a diffusion phenomenon; then, however, the approximation deteriorates again due to the presence of the convection field, which pushes the errors either towards the obstacle (Figure 3) or the bottom boundary (Figure 4).

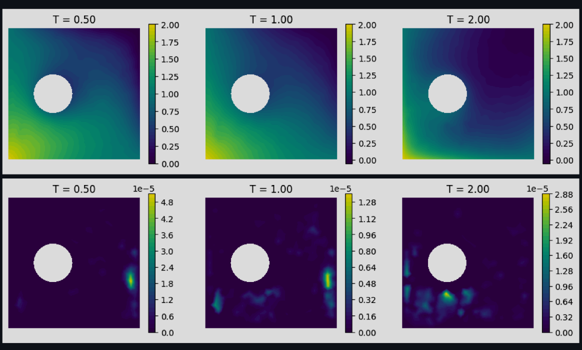

Figure 3: Test case 1, Advection-Diffusion problem. Prediction obtained for $\boldsymbol{\mu} = (0.29, 0.5)$ with the obstacle close to the source. First row: rollout prediction. Second row: RMSE related to each time step between the prediction and the corresponding ground truth solution.

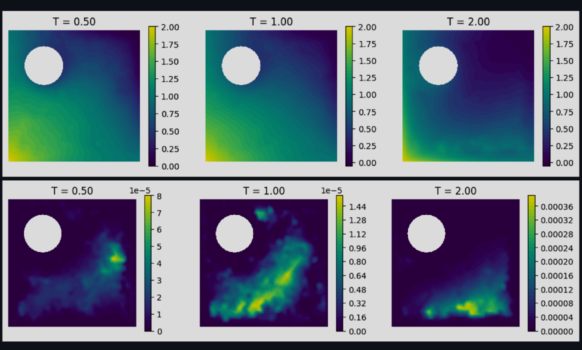

Figure 4: Test case 1, Advection-Diffusion problem. Prediction obtained for $\boldsymbol{\mu} = (0.25, 0.75)$ with the obstacle on the top left corner. First row: rollout prediction. Second row: RMSE related to each time step between the prediction and the corresponding ground truth solution.

We can further appreciate this phenomenon in Figure 5 (top row), where we have synthesized the dynamics of the relative L^2-error. More precisely, the picture shows how the quality of the approximation changes within time: to account for the variability in the test set, both median and quantile curves are reported. The trend appears to be fairly general, although the behavior quickly differentiates among different simulations (note how, as t goes from 0 to 0.25, the upper and lower quartiles rapidly split apart).

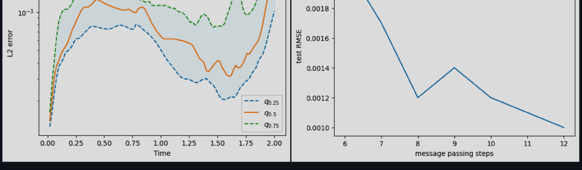

Figure 5: Test case 1, Advection-Diffusion problem. Left: L^2 relative error vs Time plot. The dashed lines represent the first and the third quantiles of the L^2 errors among all the test predictions, while the orange line is the median. The shaded area can be considered a confidence region for the simulation error. Right: Test RMSE vs message passing steps.

5.1.3 The *message passing steps* hyperparameter

Among all the hyperparameters, the one most influencing the goodness of the model is the number of message-passing steps. This number represents how much in-depth we look at the neighborhood when we propagate the message. A small number of message-passing steps may result in underfitted areas of the mesh, while a big one will slow down the training, increasing too much the number of parameters, possibly yielding overfitting.

Here, we tuned this parameter via trial and error. A plot of the corresponding results can be seen in Figure 5 (bottom row). The test RMSE reaches a local minimum for $m = 8$ message-passing steps. This is the best choice if we want to keep control of the number of total parameters of the network, which are only 61825 in this case. However, since the architecture obtained for $m = 12$ is still reasonably complex, we stick to the latter one. We do not proceed further as the improvement rate, in terms of m, no longer justifies favoring a larger number of message-passing steps.

	RMSE (mean)	**RMSE (max)**	**RMSE (min)**	t_{FOM}	t_{GNN}
Adve-diff §5.1	$1.20e-3$	$6.10e-3$	$4.0e-4$	$159.80s$	$9.83s$
Stokes 2D §5.2	$1.64e-2$	$7.35e-2$	$1.2e-3$	$115.65s$	$7.51s$
Stokes 3D §5.3	$4.37e-2$	$6.24e-2$	$1.9e-2$	$729.42s$	$10.4s$

Table 2: Comparison between FOM and GNN-surrogate in terms of model accuracy and computational time for the three case studies.

5.1.4 Generalization to obstacles with different dimensions

Problem 14 can also be extended to domains in which both the position and the dimension of the obstacle change. To this end, we modify our training dataset slightly by adding new simulations in which the obstacle has either a smaller and a larger radiusMathematically speaking, this corresponds to considering an augmented parameter space where $\boldsymbol{\mu} = (c_x, c_y, r) \in \Theta = \{(x, y) : 0 < x < 1, \;\; y \geq 0.5\} \times \{0.1, 0.15, 0.2\}$.

Again, we test the model on new simulations which have varying obstacle positions and dimensions. In Figure 6 the prediction for a new test simulation is reported. The model can generalize well on this problem even if the geometries differ a lot from each other, in terms of sizing. Moreover, there is no need to increase the number of message-passing steps, meaning that the GNN architecture has the same complexity as before.

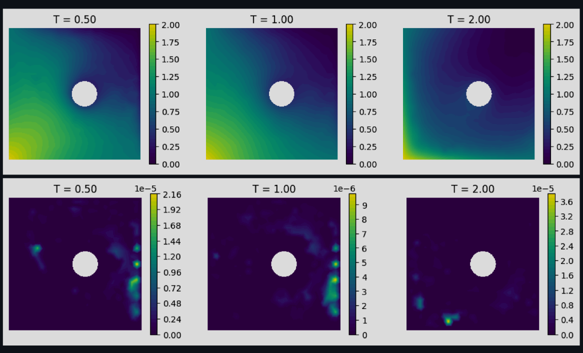

Figure 6: Test case 1, Advection-Diffusion problem. Prediction obtained for $\boldsymbol{\mu} = (0.6, 0.52, 0.1)$. First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution.

An important question that arises is whether our model can predict solutions where the obstacle has a different shape, and whether we can achieve this without having to retrain the whole network. To investigate this, we present an example in Figure 7 of a prediction obtained with a square obstacle located in the top right of the domain: strictly speaking, this configuration cannot be described in terms of our previous parametrization; nonetheless, we can still apply our GNN surrogate, as the latter only depends on the geometrical parameters through the underlying mesh (by itself, the parametrization never enters the equation).

Surprisingly, the errors are of the same order of magnitude as those discussed earlier, and the prediction of the overall dynamics is remarkably accurate. This result is attributed to the ability of the model to understand different geometries by means of its inductive structure. GNNs, in particular, can automatically incorporate the geometrical structure of the domain by utilizing both the edge connectivity matrix and the edge features. However, some difficulty is observed in handling the nodes surrounding the obstacle, especially at the corners, but this does not appear to affect the overall accuracy of the prediction. These findings suggest that our model has the potential to generalize well to other geometries, without the need for extensive retraining, thus enhancing its practical applicability in real-world scenarios.

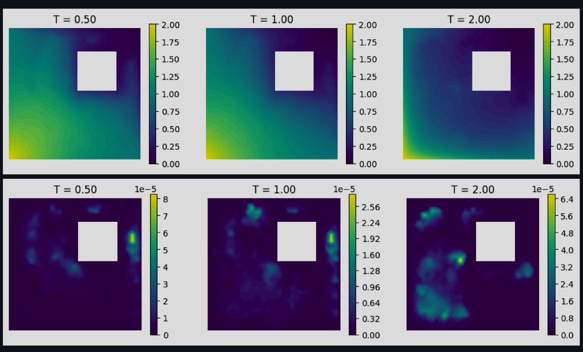

Figure 7: Test case 1, Advection-Diffusion problem. Prediction obtained for $\boldsymbol{\mu} = (0.7, 0.7, 0.3)$ with a square obstacle. First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution.

5.2 Advection-Diffusion problem in a 2D Stokes flow in proximity of a bump

We now consider another advection-diffusion problem as (14), where the advection field $\mathbf{b}$ is no longer fixed by hand, but it is rather obtained by solving the following stationary Stokes problem:

$$\begin{cases} -\nu\Delta\mathbf{b} + \nabla\, p &= 0 \text{in} \Omega \\ \nabla\cdot\mathbf{b} &= 0 \text{in} \Omega \end{cases} \tag{15}$$

where p is the pressure field and the boundary conditions are given by:

$$\mathbf{b} = 0 \;\text{ on }\; \Gamma_D, \quad \mathbf{b} = \mathbf{b_{in}} \text{on }\; \Gamma_{in}, \quad \nu\frac{\partial\mathbf{b}}{\partial\mathbf{n}} - p\mathbf{n} = 0 \;\text{ on }\; \Gamma_N,$$

with

$$\mathbf{b_{in}} = \left(\frac{40Uy(0.5-y)}{0.5^2}, 0\right), \quad U = 0.3, \quad \nu = 10^{-3}; \tag{16}$$

$\mathbf{b_{in}}$ represents the value of $\mathbf{b}$ at the inflow Γ_{in}, while Γ_D and Γ_N denote the Dirichlet wall (top and bottom) sides, and the Neumann right outflow boundaries, respectively.

This time, the domain Ω is a rectangular channel $(0,1) \times (0, 0.5)$ with a parametrized bump along the top wall edge. Here $\Gamma_{in} = \{x = 0\}$, $\Gamma_D = \{y = 0\} \cup \{y = 0.5\}$ and $\Gamma_N = \{x = 1\}$. During our simulations, we shift the position of the bump in a way that its center c_x varies from 0.35 to 0.65. Hence, we consider $\mu \in \Theta = [0.35, 0.65]$.

Regarding the advection-diffusion problem, at the inflow Γ_{in} we impose the Dirichlet boundary condition

$$u_{in}(x, y) = \frac{4y(0.5-y)}{0.5^2},$$

which is also the initial condition, while on Γ_D we impose no-slip boundary conditions and on Γ_N we set $\partial u / \partial n = 0$. The final simulation time is $T = 0.5$ and $D = 0.01$. Our results will only focus on the approximation of the solution u of the advection-diffusion problem, despite the latter also depending implicitely on (15).

5.2.1 Problem data

Our dataset is composed of 125 simulations, each obtained for a different position of the bump. In each of these cases, the mesh is rebuilt yielding a number of mesh nodes varying from 937 to 1042. The chosen time step is $\Delta t = 0.01$, resulting in 51 time snapshots for each simulation. The training set is made by 100 simulations, while the test set includes 25 simulations, both chosen randomly among the 125 FOM simulations.

Differently from our previous test case, we consider a loss function where the two terms in 11 are weighted equally (cf. Table 1). As before, we refer to Table 1 for further details about GNN and training hyperparameters.

5.2.2 Numerical results

As before, quantitative results are in Table 2. In this more complex problem, the RMSEs are higher than the ones obtained in the previous example; however, the predictions are still fairly accurate and we still outperform the ground truth solver in terms of time efficiency.

Indeed, as shown in Figures 8-9, the predicted dynamics is still very accurate and we do not spot any propagation error. The prediction seems to get worse at some nodes which are either close to the bump or to the upper edge, in which we have imposed the no-slip conditions. Conversely, the errors in proximity of the inflow are higher at initial times, but they tend to fade out as the simulation evolves (this is true also for our worst simulation, Figure 9).

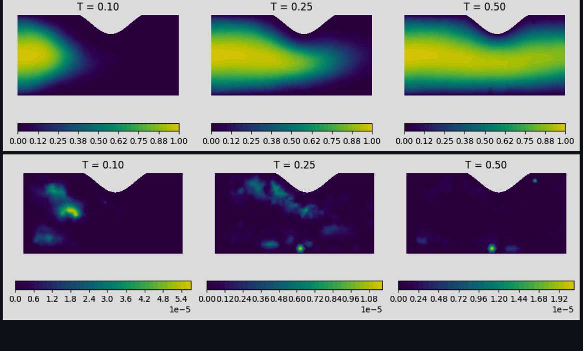

Figure 8: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. Prediction obtained for $\mu = 0.58$ with the bump on the right part of the upper edge. First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution

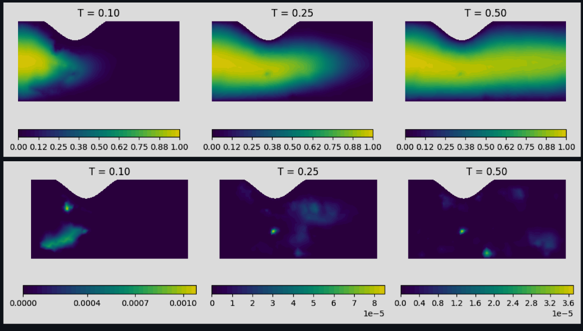

Figure 9: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. Worst case scenario: bump close to the inflow ($\mu = 0.355$). First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution

Our qualitative considerations are also supported by the plot in Figure 10, which reports the behavior in time of the L^2 relative error between predictions and FOM solutions. Clearly, the first time instants are the most challenging ones, as that is when the inflow and the bump position determine the dynamics of the system. Overall, we highlight that the model is able to self-adjust, since errors tend to decrease as the simulation time evolves, also showing some degree of robustness to the possible presence of noise during the simulation.

Of note, these considerations hold uniformly over the test set, as clearly indicated by the width of the quantile bands. This is a desirable property since real-world problems often have some degree of uncertainty or noise, and a model that can handle different scenarios is more likely to be useful in practice.

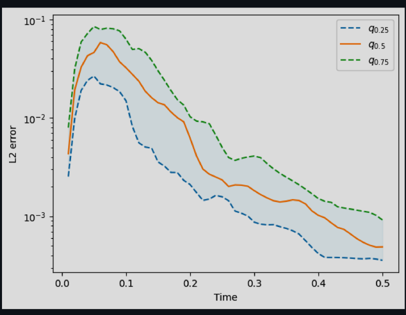

Figure 10: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. L^2 relative error vs Time plot: The dashed lines represent the first and the third quantiles of the L^2 errors among all the test predictions, while the orange line is the median. The shaded area can be seen as a confidence region for the simulation error.

5.2.3 Generalization to bumps with different positions and dimensions

This example can be generalized by letting the bump vary its dimension and possibly switch from the upper to the lower edge. Hence, we consider a new dataset consisting of 185 simulations in which the height of the bump is allowed to change, $h \in \{0.08, 0.12, 0.175\}$, and its center can vary along both the upper and lower edge in the interval $[0.4, 0.6]$, respectively. In other words, the new geometrical parameters are $\boldsymbol{\mu} = (c_x, c_y, h) \in \Theta := [0.4, 0.6] \times \{0., 0.5\} \times \{0.08, 0.12, 0.175\}$.

The results show that the implemented GNN-based model is able to learn correctly the geometry of the problem even if we the domain varies substantially within the dataset. For instance, in Figure 11 the height of the bump influences a lot the system dynamics, however the network correctly infers the behavior of the flow around the obstacle. Here, the bump has height $h = 0.175$ and is located at the lower edge with center at $x = 0.453$, that is $\boldsymbol{\mu} = (0.453, 0, 0.175)$. The height of the bump has a significant impact on the accuracy of model prediction, particularly near the upper edge of the domain. Errors that arise in this region can propagate throughout the domain, affecting the accuracy of predictions at other locations as well. However, the self-adjustment mechanism of the model is effective in mitigating these errors as they propagate toward the outflow, resulting in improved accuracy in this region. Overall, the model ability to account for the influence of the bump height on the flow dynamics contributes to its strong predictive performance.

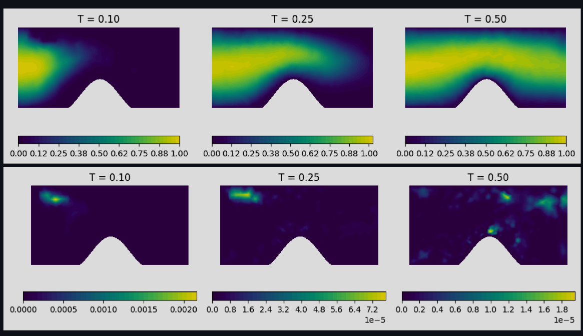

Figure 11: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. Prediction obtained for $\boldsymbol{\mu} = (0.453, 0, 0.175)$ with the bump in the lower edge with center at $x = 0.453$ and height $h = 0.175$. First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution.

In Figure 12 the bump has height $h = 0.08$ and is located at the upper edge with center at $x = 0.467$, that is $\boldsymbol{\mu} = (0.467, 0.5, 0.08)$. The accuracy of the model predictions decreases when the size of the bump is smaller. This is primarily due to the fact that, as the size of the domain increases, so does the number of nodes, making the inference process more challenging. In this problem, the number of nodes varies from 936 to 1054, which is a wide range for unstructured meshes and geometries that differ significantly from each other. As a result, error propagation is more significant in this case compared to the other examples. This is highlighted by the persistence of relatively large errors at $T = 0.25$, despite the overall dynamics being well-predicted by the model. This suggests that the model can effectively capture the underlying physics of the system, even in cases where the inference is more challenging due to the higher number of nodes. In general, the L^2-errors exhibit the same behavior as before: see Figure 13 in comparison with Figure 10.

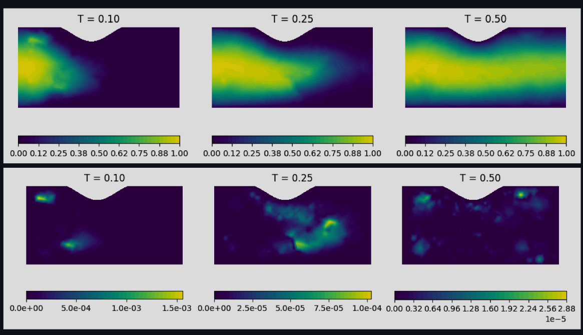

Figure 12: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. Prediction with bump in the lower edge with center at $x = 0.453$ and height $h = 0.08$ ($\boldsymbol{\mu} = (0.467, 0.5, 0.08)$). First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution.

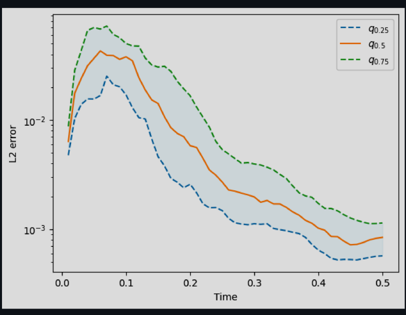

Figure 13: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. L^2 relative error vs Time plot.

We can further test the robustness of our model by evaluating its ability to predict solutions in channels with varying shapes of the bump, without requiring any retraining. Figure 14 displays the prediction results of a simulation with a triangular bump located on the upper edge. In addition to the fact that the errors are of the same order of magnitude as previously discussed, the overall dynamics is, once again, accurately predicted. However, the regularity of the solution poses some difficulty for the model. Nonetheless, this does not appear to significantly impact the accuracy of the prediction.

This example highlights the flexibility of graph neural networks in handling simulations with variable geometries and limited training data, while still producing reliable results.

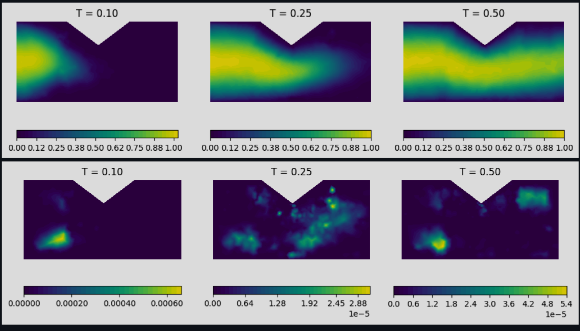

Figure 14: Test case 2, Advection-Diffusion problem in a 2D Stokes flow. Prediction with a triangular bump on the upper edge. First row: rollout prediction. Second row: RMSE related to each timestep between the prediction and the corresponding ground truth solution.

5.3 Advection-Diffusion problem in a 3D Stokes flow around a cylinder

To further increase the problem difficulty, we finally consider the same problem discussed in Section 5.2, now set in a 3D domain obtained by an extrusion on the z-axis of the rectangle $R = (0, 1) \times (0, 0.5)$ with a cylindrical hole

$$C = \{(x, y) : \ (x - c_x)^2 + (y - c_y)^2 \leq (0.05)^2\}.$$

We let the position of the obstacle vary as $\mu = (c_x, c_y) \in \Theta := [0.2, 0.4] \times [0.2, 0.3]$. This time, we exploit the FOM to generate 150 different simulations, 125 for training and 25 for testing.

5.3.1 Problem data

The mesh nodes of the simulations vary from 1353 to 1542, thus increasing the complexity of the problem with respect to the other examples we have discussed so far. We implement the approach following the same ideas adopted for the previous test cases: we refer to Table 1 for further details about the network design and the training hyperparameters.

In this regard, we have made some minor modifications to account for the increased complexity entailed by the presence of a three-dimensional geometry. These concern: an increased number of message-passing steps (to better cover the spatial domain), an increased noise variance (to further improve the stability of our simulations during rollout), and a reduced number of epochs (to avoid overfitting). By adopting this training strategy, we aim to strike a balance between model accuracy and computational efficiency while still being able to capture the complex dynamics of the system.

5.3.2 Numerical Results

The results of the rollout predictions of the test simulations are summarized in Table 2. Clearly, the higher error obtained in this example is due to the increased complexity of the problem. However, it is noteworthy that despite the higher error, there is a significant improvement in time complexity. Once trained, our model can be up to two orders of magnitude faster compared to the FOM solver. This reduction in time complexity can lead to faster and more efficient simulations, which is particularly important for time-critical applications or when a large number of simulations are required. Therefore, despite the slightly higher error, our model can still provide a significant advantage in terms of time and computational resources.

Upon examining the predictions in greater detail, as shown in Figure 15, a comparison can be made between the prediction of a simulation with the obstacle positioned centrally, and its corresponding ground truth solution. It is evident that the simulation deteriorates as it progresses toward the outflow. Unlike the 2D case, self-adjustment is not observed in this scenario, as the nodes located to the right of the obstacle are heavily influenced by its position. This may result in some values being underestimated in the prediction, particularly in the tail of the flow. Unfortunately, this is a known drawback of GNNs, as deep architectures tend to oversmooth predictions. Therefore, even if the dynamics are predicted accurately, node values may be more dispersed. Furthermore, this problem is exacerbated by an increase in the number of message passing steps, which, in this example, are necessary for an acceptable prediction.

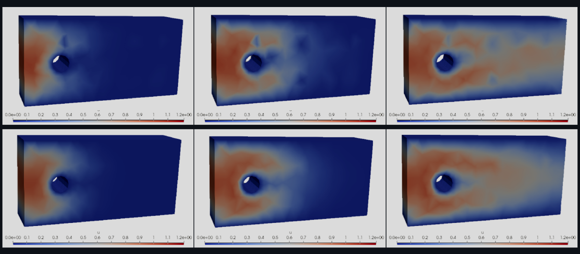

Figure 15: Test case 3, Advection-Diffusion problem in a 3D Stokes flow. Prediction obtained for $\mu = (0.29, 0.25)$. 3 time steps of simulation. First row: Rollout prediction. Second row: ground truth solution.

Nevertheless, as illustrated in Figure 16, modifying the position of the obstacle does not significantly affect the overall accuracy of the solution. Despite the aforementioned issues, the flow pattern is captured correctly, and no propagation of the errors is observed. Of remarkable importance is the consistently accurate prediction in the proximity of the obstacle, which is always a critical aspect to be predicted. This observation underscores the model ability to learn the geometrical properties of the problem while preserving the graph structure of the mesh. Therefore, these results suggest that the model is sufficiently robust in predicting flow patterns in various configurations, and can generalize well to other geometries.

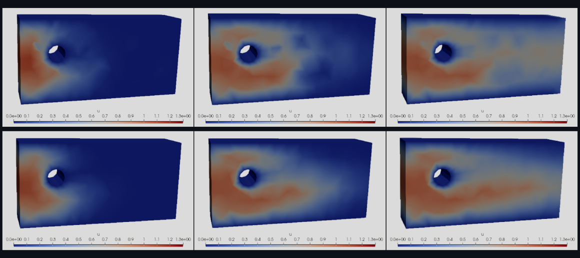
0.0e+00 0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1 1.1 1.2 1.3e+00
u

Figure 16: Test case 3, Advection-Diffusion problem in a 3D Stokes flow. Prediction obtained for $\mu = (0.23, 0.3)$. 3 time steps of simulation. First row: Rollout prediction. Second row: ground truth solution.

Upon observing the L^2 relative error plot on the test set in Figure 17, we can draw quantitative conclusions regarding the previously discussed results. The plot indicates that the test error has an appropriate upper bound and that it increases significantly during the first few time steps, which is consistent with the observed prediction behavior. After the initial increase, the error gradually decays, showing that the model has learned the underlying dynamics of the system. However, towards the end of the simulation, we observe a slight increase in the error, which is coherent with what we have previously mentioned about the tendency of these architectures to dispersion. This behavior may be due to the accumulation of errors during the long-term prediction. Therefore, we can conclude that while the GNN-based model shows promising results, there is still room for improvement in terms of accuracy and robustness.

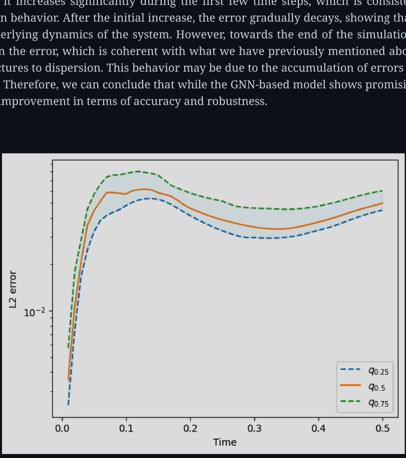

Figure 17: Test case 3, Advection-Diffusion problem in a 3D Stokes flow. L^2 relative error vs Time plot: The dashed lines represent the first and the third quantiles of the L^2 errors among all the test predictions, while the orange line is the median. The shaded area can be seen as a confidence region for the simulation error.

5.4 Comparison with Feed Forward Neural Networks

Feed Forward Neural Networks (FFNNs) are usually employed for building reduced-order models because they have the capability to capture strong nonlinearity through their fully connected structure [22, 28, 24]. As we explained in the introduction, however, using FFNNs is not straightforward when dealing with geometric variability, as these models require fixing both the input and output dimension. We recall, in fact, that FFNN architectures are nothing but MLP units. More precisely, any FFNN model of depth $s \geq 1$ is a map of the form

$$\Psi := L_{s+1} \circ L_s \circ \cdots \circ L_1, \tag{17}$$

where $L_i : \mathbb{R}^{n_i} \to \mathbb{R}^{n_{i+1}}$ are nonlinear maps (layers) operating as

$$L_i : \mathbf{v} \mapsto \rho_i(\mathbf{W}_i\mathbf{v} + \mathbf{b}_i),$$

where $\rho_i : \mathbb{R} \to \mathbb{R}$ is the activation function (acting componentwise), while $\mathbf{W}_i$ and $\mathbf{b}_i$ are the trainable parameters (weights and biases, respectively). Compared to GNNs, these computational units are substantially less flexible, as they can only accept inputs of a specific dimension (here, n_1), and will always produce outputs of a given size (here, n_{s+2}). Consequently, it would be impossible to implement a model such as (10) using naive FFNNs, at least not in the case of parametrized domains.

Still, it is true that we could circumvent this problem by interpolating each PDE solution over a fixed rectangular grid, with the convention that $u_\mu(\mathbf{x}) = 0$ if $\mathbf{x} \notin \Omega_\mu$. Then, it would be possible -in principle- to replicate the same construction proposed in Section 4, but only using dense architectures such as (17). Our claim, however, is that without the *relational inductive bias* of GNNs, these model have no hope of generalizing over unseen geometries. To prove this, we shall compare the performances obtained by using either FFNNs or GNNs in (10) on the two examples of Sections 5.1 and 5.2. In particular, to simplify, we consider for each problem the following datasets:

- for the first Advection-Diffusion problem, we let the obstacle vary only in its position, resulting in 100 random simulations (80 for training and 20 for testing) ;
- for the Advection-Diffusion problem in a 2D Stokes flow, we let the bump vary in its position along the upper and lower edges but not in its height, resulting in 125 random simulations (100 for training and 25 for testing).

In order to train the FFNNs, we map all the simulations onto a common rectangular grid consisting of 128×128 vertices. We then repeat the same construction presented in Section 4, up to replacing the GNN modules with MLP layers. Since FFNNs tend to overfit if trained for a long time, we train the model for 500 epochs, using the same learning rate and loss weights as the ones described in Sections 5.1 and 5.2.

Results are in Figures 18-20. As testified by the boxplots, the quality of FFNN predictions can change a lot from case to case; conversely, GNN-surrogates are much more stable and report errors with a smaller variability.

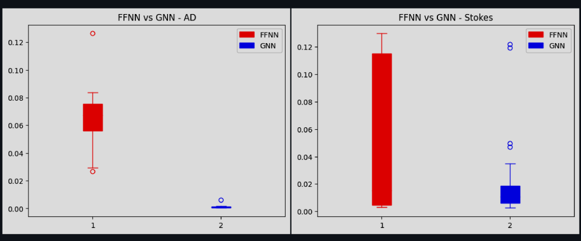

Figure 18: Boxplots of the RMSEs of the two predictions

Two examples of FFNN predictions on test set simulations, together with the corresponding GNN predictions, are shown in Figures 19 and 20. The predictions are done using the same values of geometrical parameters in order to highlight the different performances in the generalization on unseen domains. Here, the difference between the two approaches becomes evident. While FFNNs may capture the overall dynamics of the system quite well, they fail at understanding the geometrical properties of the solution, which ultimately makes them unable to generalize (see, e.g., Figure 20, where the FFNN clearly ignores the actual location of the bump).

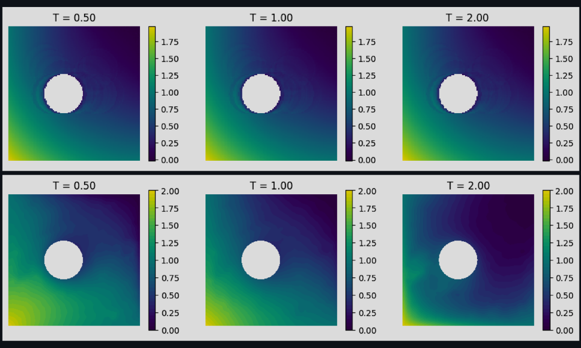

Figure 19: Test case 1, Advection - Diffusion problem. Comparison between FFNN and GNN prediction for $\boldsymbol{\mu} = (c_x, c_y) = (0.4, 0.5)$. First row: FFNN prediction. Second row: GNN prediction.

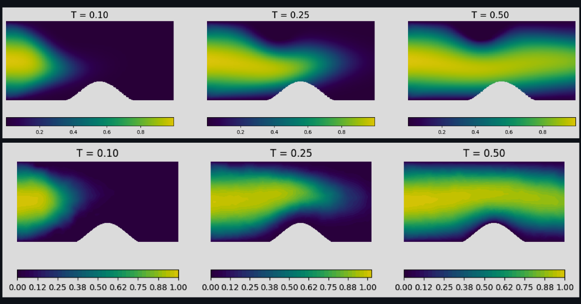

Figure 20: Test case 2, Advection - Diffusion problem in a 2D Stokes flow. Comparison between FFNN and GNN prediction for $\boldsymbol{\mu} = (c_x, c_y) = (0.6, 0)$. First row: FFNN prediction. Second row: GNN prediction.

Another key aspect to analyze is the number of model parameters. In fact, due to their fully connected structure, the complexity of FFNNs can increase dramatically with the problem complexity, implying a higher tendency to overfitting (thus the need of more training data) and less scalable models.

A possible strategy to overcome both these issues could be to rely on grid-based models, such as Convolutional Neural Networks (CNNs). In fact, these models can reduce the number of parameters by sharing them, which helps to mitigate the overfitting issue. However, while CNNs might be able to capture the dynamics of the system, they would still ignore the geometric structure of the problem, which makes them unsuitable for complex geometries. Similarly, alternative approaches such as Mesh-Informed Neural Networks (MINNs) [11] do not provide a comprehensive solution, as they can only tackle one geometry at a time.

6 Conclusions

We presented a novel approach to surrogate modeling based on Graph Neural Networks (GNNs) for the efficient evolution of dynamical systems defined over parameter dependent spatial domains. The approach differs substantially from classical Reduced Order Modelling techniques, in that it provides a way to handle parameter dependent PDEs with a variable number of degrees of freedom. The method is based on a data-driven time-stepping scheme that explicitly accounts for the Markovian structure of the dynamical system, while also including geometric information via GNN modules. The approach is shown capable of yielding stable simulations, even for long rollouts, while simultaneously generalizing to unseen geometries, thus providing remarkable benefits when compared to other techniques based on different neural network architectures.

Although limited to fairly simple problems, our results indicate that GNNs can be a valuable tool for ROM practitioners, providing researchers with new ways for handling geometric variability. Future research may involve the exploration of hybrid approaches where GNNs are combined with other well-established Deep Learning-based reduced order models, such as autoencoders and U-Net-like architectures, in an attempt to generalize the whole idea to more complicated problems with thounsands or millions of degrees of freedom.

Another interesting question could be whether this approach can benefit from the integration of suitable attention mechanisms, or other forms of neural network architectures, that can selectively weight the contributions of different nodes in the graph. We leave these considerations for future work.

Acknowledgments

The present research is part of the activities of project Dipartimento di Eccellenza 2023-2027, funded by MUR, and of project FAIR (Future Artificial Intelligence Research) project, funded by the NextGenerationEU program within the PNRR-PE-AI scheme (M4C2, Investment 1.3, Line on Artificial Intelligence). NF, SF and AM are members of Gruppo Nazionale per il Calcolo Scientifico (GNCS) and of Istituto Nazionale di Alta Matematica (INdAM).

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

Author Declarations

The authors have no conflicts to disclose.

References

[1] David Amsallem and Charbel Farhat.
An online method for interpolating linear parametric reduced-order models.
SIAM Journal on Scientific Computing, 33(5):2169–2198, 2011.

[2] Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, and Koray kavukcuoglu.
Interaction networks for learning about objects, relations and physics.
In *Proceedings of the 30th International Conference on Neural Information Processing Systems*, NIPS'16, page 4509–4517, Red Hook, NY, USA, 2016. Curran Associates Inc.

[3] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al.
Relational inductive biases, deep learning, and graph networks.
arXiv preprint arXiv:1806.01261, 2018.

[4] Andrea Bonito, Albert Cohen, Ronald DeVore, Diane Guignard, Peter Jantsch, and Guergana Petrova.
Nonlinear methods for model reduction.
ESAIM: Mathematical Modelling and Numerical Analysis, 55(2):507–531, 2021.

[5] Simone Brivio, Stefania Fresca, Nicola Rares Franco, and Andrea Manzoni.
Error estimates for pod-dl-roms: a deep learning framework for reduced order modeling of nonlinear parametrized pdes enhanced by proper orthogonal decomposition.
arXiv preprint arXiv:2305.04680, 2023.

[6] Gergely Buza, Shobhit Jain, and George Haller.
Using spectral submanifolds for optimal mode selection in nonlinear model reduction.
Proceedings of the Royal Society A, 477(2246):20200725, 2021.

[7] Kevin Carlberg.
Adaptive h-refinement for reduced-order models.
International Journal for Numerical Methods in Engineering, 102(5):1192–1210, 2015.

[8] Ludovica Cicci, Stefania Fresca, and Andrea Manzoni.
Deep-hyromnet: A deep learning-based operator approximation for hyper-reduction of nonlinear parametrized pdes.
Journal of Scientific Computing, 93(2):57, 2022.

[9] Federico Fatone, Stefania Fresca, and Andrea Manzoni.
Long-time prediction of nonlinear parametrized dynamical systems by deep learning-based reduced order models.
arXiv preprint arXiv:2201.10215, 2022.

[10] Nicola Franco, Andrea Manzoni, and Paolo Zunino.
A deep learning approach to reduced order modelling of parameter dependent partial differential equations.
Mathematics of Computation, 92(340):483–524, 2023.

[11] Nicola Rares Franco, Andrea Manzoni, and Paolo Zunino.
Learning operators with mesh-informed neural networks.
arXiv preprint arXiv:2203.11648, 2022.

[12] Stefania Fresca, Luca Dede', and Andrea Manzoni.
A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized pdes.
Journal of Scientific Computing, 87:1–36, 2021.

[13] Stefania Fresca and Andrea Manzoni.
Pod-dl-rom: enhancing deep learning-based reduced order models for nonlinear parametrized pdes by proper orthogonal decomposition.
Computer Methods in Applied Mechanics and Engineering, 388:114181, 2022.

[14] Rini Jasmine Gladstone, Helia Rahmani, Vishvas Suryakumar, Hadi Meidani, Marta D'Elia, and Ahmad Zareei.

Gnn-based physics solver for time-independent pdes.
arXiv preprint arXiv:2303.15681, 2023.

[15] Mengwu Guo and Jan S Hesthaven.
Data-driven reduced order modeling for time-dependent problems.
Computer methods in applied mechanics and engineering, 345:75–99, 2019.

[16] Mengwu Guo, Shane A McQuarrie, and Karen E Willcox.
Bayesian operator inference for data-driven reduced-order modeling.
Computer Methods in Applied Mechanics and Engineering, 402:115336, 2022.

[17] W. L. Hamilton, R. Ying, and J. Leskovec.
Inductive representation learning on large graphs.
Advances in Neural Information Processing Systems, pages 1024–1024, 2017.

[18] W. L. Hamilton, R. Ying, and J. Leskovec.
Inductive representation learning on large graphs.
CoRR, abs/1706.02216, 2017.

[19] Martin Hess, Alessandro Alla, Annalisa Quaini, Gianluigi Rozza, and Max Gunzburger.
A localized reduced-order modeling approach for pdes with bifurcating solutions.
Computer Methods in Applied Mechanics and Engineering, 351:379–403, 2019.

[20] Jan S Hesthaven, Cecilia Pagliantini, and Nicolò Ripamonti.
Rank-adaptive structure-preserving model order reduction of hamiltonian systems.
ESAIM: Mathematical Modelling and Numerical Analysis, 56(2):617–650, 2022.

[21] Jan S Hesthaven, Gianluigi Rozza, Benjamin Stamm, et al.
Certified reduced basis methods for parametrized partial differential equations, volume 590.
Springer, 2016.

[22] Jan S Hesthaven and Stefano Ubbiali.
Non-intrusive reduced order modeling of nonlinear problems using neural networks.
Journal of Computational Physics, 363:55–78, 2018.

[23] Masanobu Horie and Naoto Mitsume.
Physics-embedded neural networks: Graph neural pde solvers with mixed boundary conditions.
Advances in Neural Information Processing Systems, 35:23218–23229, 2022.

[24] Sean T Kelly and Bogdan I Epureanu.
Data-driven reduced-order model for turbomachinery blisks with friction nonlinearity.
In *Nonlinear Structures & Systems, Volume 1: Proceedings of the 40th IMAC, A Conference and Exposition on Structural Dynamics 2022*, pages 97–100. Springer, 2022.

[25] D. P. Kingma and J. Ba.
Adam: A method for stochastic optimization.
International Conference on Learning Representations (ICLR), 2015.

[26] T.N. Kipf and M. Welling.
Semi-supervised classification with graph convolutional networks.
5th International Conference on Learning Representations (ICLR-17), 2016.

[27] Mingwu Li, Shobhit Jain, and George Haller.
Model reduction for constrained mechanical systems via spectral submanifolds.
Nonlinear Dynamics, 111(10):8881–8911, 2023.

[28] Nikolaj T Mücke, Sander M Bohté, and Cornelis W Oosterlee.
Reduced order modeling for parameterized time-dependent pdes using spatially and memory aware deep learning.
Journal of Computational Science, 53:101408, 2021.

[29] Fionn Murtagh.
Multilayer perceptrons for classification and regression.
Neurocomputing, 2(5-6):183–197, 1991.

[30] Federico Negri, Gianluigi Rozza, Andrea Manzoni, and Alfio Quarteroni.
Reduced basis method for parametrized elliptic optimal control problems.
SIAM Journal on Scientific Computing, 35(5):A2316–A2340, 2013.

[31] Mathias Niepert, Mohamed Ahmed, and Konstantin Kutzkov.
Learning convolutional neural networks for graphs.
In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 2014–2023, New York, New York, USA, 20–22 Jun 2016. PMLR.

[32] Cecilia Pagliantini.
Dynamical reduced basis methods for hamiltonian systems.
Numerische Mathematik, 148(2):409–448, 2021.

[33] Luca Pegolotti, Martin R Pfaller, Natalia L Rubio, Ke Ding, Rita Brugarolas Brufau, Eric Darve, and Alison L Marsden.
Learning reduced-order models for cardiovascular simulations with graph neural networks.
arXiv preprint arXiv:2303.07310, 2023.

[34] Tobias Pfaff, Meire Fortunato, Alvaro Sanchez-Gonzalez, and Peter W Battaglia.
Learning mesh-based simulation with graph networks.
arXiv preprint arXiv:2010.03409, 2020.

[35] Federico Pichi, Beatriz Moya, and Jan S Hesthaven.
A graph convolutional autoencoder approach to model order reduction for parametrized pdes.
arXiv preprint arXiv:2305.08573, 2023.

[36] A. Quarteroni and A. Valli.
Numerical approximation of partial differential equations, volume 23.
Springer, 01 1994.

[37] Alfio Quarteroni, Andrea Manzoni, and Federico Negri.
Reduced basis methods for partial differential equations: an introduction, volume 92.
Springer, 2015.

[38] Francesco Romor, Giovanni Stabile, and Gianluigi Rozza.
Non-linear manifold reduced-order models with convolutional autoencoders and reduced over-collocation method.
Journal of Scientific Computing, 94(3):74, 2023.

[39] D. Rumelhart, G. Hinton, and R. Williams.
Learning representations by back-propagating errors.
Nature, pages 533–536, 1986.

[40] T Konstantin Rusch, Michael M Bronstein, and Siddhartha Mishra.
A survey on oversmoothing in graph neural networks.
arXiv preprint arXiv:2303.10993, 2023.

[41] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia.
Learning to simulate complex physics with graph networks.
In *International conference on machine learning*, pages 8459–8468. PMLR, 2020.

[42] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini.
The graph neural network model.
IEEE transactions on neural networks, 20(1):61–80, 2008.

[43] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjing Wang, and Yu Sun.
Masked label prediction: Unified message passing model for semi-supervised classification.
In Zhi-Hua Zhou, editor, *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI-21*, pages 1548–1554. International Joint Conferences on Artificial Intelligence Organization, 8 2021.
Main Track.

[44] Jonathan Shlomi, Peter Battaglia, and Jean-Roch Vlimant.
Graph neural networks in particle physics.
Machine Learning: Science and Technology, 2(2):021001, 2020.

[45] Khemraj Shukla, Mengjia Xu, Nathaniel Trask, and George E Karniadakis.
Scalable algorithms for physics-informed neural and graph networks.
Data-Centric Engineering, 3:e24, 2022.

[46] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio.
Graph attention networks.
6th International Conference on Learning Representation, 2017.

[47] Si Zhang, Hanghang Tong, Jiejun Xu, and Ross Maciejewski.
Graph convolutional networks: a comprehensive review.
Computational Social Networks, 6(1):1–23, 2019.

Copyright Privacy Policy Generated by LaTeXML oxide